



Monitor Your Syncs and Backups
Get Status of Syncs via Logs, Email, Push
Notification or System Notification

ChronoSync 4.8
Download the Free Trial

[Home](#) [Magazine](#) [Live Events](#) [News](#) [Forums](#) [Store](#) [Directory](#) [Editorial](#) [Advertising](#) [User/Login](#) [Contact](#) [Talk to us](#)

You are here



Volume Number: 9

Issue Number: 6

Column Tag: Programmers' Challenge

Programmers' Challenge

By Mike Scanlin, MacTech Magazine Regular Contributing Author

 Note: [Source code files](#) accompanying article are located on MacTech CD-ROM or source code disks.

WHERE IN THE WORLD?

This month's Challenge is a real world problem previously solved without reals by real MacTech reader Allen Stenger (Gardena, CA): You are given a large array of unique city names in alphabetical order ending with a sentinel of 'ZZZ' and a single city name to find in this array. You are to return the index of the desired city, or if it doesn't match exactly, a list of the five closest matches. If there are more than five closest ones, return any five of them. The closeness-of-match algorithm, for purposes of this challenge, is "differing in the fewest positions from the desired name." Take the longest of the two strings you're comparing and each time a position doesn't match (or is beyond the end of the shorter string) add 1 to your difference count; the smallest difference count is considered closest.

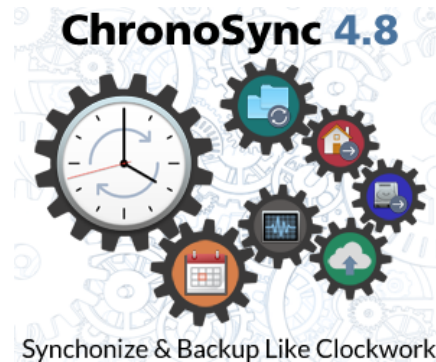
The input city list is an array of Str255s (pascal strings) in one big block where there will be, say, 200 to 1000 unique city names (in alphabetical order, all uppercase, with the 'ZZZ' sentinel as the last element). The city you are to find is the uppercase string pointed to by cityToFindNamePtr. The prototype of the function you write is:

```
Boolean FindCity(cityNames,
    cityToFindNamePtr, closestMatches)
Str255    cityNames[];
Str255    *cityToFindNamePtr;
unsigned short    closestMatches[];
```

If there is an exact match, return TRUE and set closestMatches[0] equal to the zero-based index of the match. If there is not an exact match, return FALSE and set closestMatches[0..4] to the five closest matches (in order of closeness, or, if they are equally close, in any order).

Allen says that this problem came up while working on the American Airlines reservation system, SABRE, a long time ago. He says that his tests then showed that city names were

Showcase



THIS MONTH'S ISSUE



Issue 353

*Code Debugging and
Spelunking*

Check out the latest including:
x86 Code Debugging and
Spelunking ; An Introduction
to WebGL ; Managing ESXi
from your Mac ; Deploying
SHA-3 ; An introduction to
Julia ; KoolTools: BBEdit 11.5,
TextSoap For OS X, Honeywell
Lyric Round Wi-Fi Thermostat,
My Cloud EX2 Ultra ; MacTech
Spotlight: Daniel H Steinberg

misspelled about 1/3rd of the time due to either poor spelling or typing by the reservation agents (no offense intended). You can use that information in your solution if you like; I will be simulating it with test data (exact matches will occur only 33% of the time I call your routine from my test bench).

TWO MONTHS AGO WINNER

Congratulations to Steve Israelson (Vancouver, B.C.) for winning the "Rotated Bits" challenge. Right on his heels is Andy Scheck (location unknown) with an entry that was half as many bytes but which was just a little too slow in some cases to win. Jeff Mallett (Hickory, NC) deserves praise for his extremely small solution which was also near the top in terms of speed. Because it is so small, it is listed here as well, after Steve's winning solution.

There were 16 entries to this challenge. Two of them were disqualified immediately because they were in assembly (nice try) and six others were disqualified for giving incorrect results (try testing with a 9x17 bitmap). Many people e-mailed me and asked if they could assume that rowBytes was divisible by four. The answer is: no. However, you could always case out on that case early on in your routine and then call one of two rotate functions based on if (rowBytes % 4 == 0) or not. Just so long as your solution has one entry point you can do whatever you like inside.

Here's a summary of the eight entries that gave correct results. The bytes column is the code size, test 1 is the ticks to rotate 4000 small bitmaps (20x30 and smaller) and test 2 is the ticks to rotate 400 medium size bitmaps (300x400 pixels, approx). In all cases, the bitmaps were filled with the same random data before rotating:

Name bytes test 1 test 2

Steve Israelson 740 72 638

Andy Scheck 386 75 778

Jeff Mallett 194 91 936

Patrick Breen 668 125 902

Stepan Riha 656 122 1186

Dave Darrah 362 138 1453

Jan Bruyndonckx 490 205 1785

Dominic Mazzoni 244 630 6687

Steve's winning solution is reasonably commented so I won't go into detail here describing how it works. I will, however, offer some peephole optimizations to make it even better.

The first thing that the optimized eye notices about his code is that he multiplies, divides and mods by 8. It's a dangerous assumption to make that the compiler will be smart and substitute shifts and ands where it can. I always prefer to make them explicit. Thus, I would change these lines:

```
srcRowX8 = srcRowBytes * 8;
srcData = const1 + srcX / 8;
if (remainder = (srcWidth % 8))
```

to these:

```
srcRowX8 = srcRowBytes << 3;
srcData = const1 + (srcX >> 3);
if (remainder = (srcWidth & (8 - 1)))
```

See Now



Search for

Yahoo! Search

TODAY'S DEAL

In addition to those, there is one point in his code where he multiplies by 7. He'd be better off by multiplying by 8 (using a shift) and then subtracting one. This:

```
src += srcOffset * 7;
is faster as:
```

```
src += srcOffset << 3;
src -= srcOffset;
```

After I made these changes and then ran test 2, the time went from 638 ticks to 542 ticks (it reduced the code size by 18 bytes, too).

Here's Steve's winning solution, followed by Jeff's tiny solution:

```
/*-----
Steve Israelson
Motion Works Corp.
-----*/
void RotateBitMapClockwise(BitMap *, BitMap *);
void rotateBlock(unsigned long *,unsigned char *,
    long,unsigned char *,long);
void rotateLimitedBlock(unsigned long *,
    unsigned char *, long,unsigned char *,long,
    short);

/*-----
Rotate the bitmap from src to dst by +90 degrees. Make
a table for use in the rotate block routine. Step through
the src in 8x8 blocks, starting from the bottom left. Step
through dst in 8x8 blocks, starting from the top left.
Handle the right edge of the bitmap specially to prevent bits
from being rotated into memory outside of the destination.
-----*/
void RotateBitMapClockwise(BitMap *src, BitMap *dst)
{
    unsigned long table[16] =
    {0x00000000, 0x00000001, 0x00000100, 0x00000101,
    0x00010000, 0x00010001, 0x00010100, 0x00010101,
    0x01000000, 0x01000001, 0x01000100, 0x01000101,
    0x01010000, 0x01010001, 0x01010100, 0x01010101};
    /* x,y co-ord of source block */
    register short srcX, srcY;
    /* row bytes, dereferenced for speed */
    short srcRowBytes, dstRowBytes;
    /* src rowbytes * 8, for speed */
    short srcRowX8;
    /* width and height of src */
    short srcWidth, srcHeight;
    /* pointer to src and dst 8x8 blocks */
    char *srcData, *dstData;
    /* pointer to bitmaps de-referenced for speed */
    char *srcAddr, *dstAddr;
    /* last few pixels to do specially */
    short remainder;
    /* a constant, pre-calculated for speed */
    char *const1;
```

A promotional banner for DigitalOcean SSD Virtual Servers. It features the DigitalOcean logo at the top. Below it, the text "SSD Virtual Servers" is prominently displayed in orange. To the left, a price of "\$5 /mo." is shown. In the center, "20GB SSD Disk" is listed. To the right, "512MB Memory" is shown. A large orange button with the text "GET STARTED" and a right-pointing arrow is centered below these details. At the bottom, a smaller line of text says "Launch a server in 55 seconds!".

Search



Community Search:

GO

MacTech Search:

GO

Software Updates via MacUpdate

[Lyn 2.2.1 - Lightweight image browser an...](#)

[Minecraft 1.19.3 - Popular sandbox build...](#)

[Boom 3D 1.4.0 - \\$19.99](#)

[Visual Studio Code 1.74.0 - Cross-platfo...](#)

[Notion 2.1.9 - A unified workspace for m...](#)

[Spotify 1.2.0.1165 - Stream music, creat...](#)

[Thunderbird 102.5.1 - Email client from...](#)

[Pinegrow 7.03 - Mockup and design web pa...](#)

[Adobe After Effects 2022 23.1 - Create p...](#)

[SteerMouse 5.6.7 - Powerful third-party...](#)

SEE ALL

An advertisement for Drivesavers Data Recovery. It is split into two vertical panels. The left panel has a dark blue background with the text "Remember when typing felt good?" in white. The right panel has a red background with a white circular logo that says "DRIVESAVERS" and "DATA RECOVERY" around a central graphic. Below the logo, the phone number "800.440.1904" and the website "www.drivesavers.com" are listed in white text.

Latest Forum Discussions

SEE ALL

```
/* de-reference some constants */
srcRowBytes = src->rowBytes;
dstRowBytes = dst->rowBytes;
srcAddr = src->baseAddr;
dstAddr = dst->baseAddr;

/* calculate some values */
srcWidth = src->bounds.right - src->bounds.left;
srcHeight = src->bounds.bottom - src->bounds.top;

srcRowX8 = srcRowBytes * 8;
const1 = srcAddr + (srcHeight - 8) * srcRowBytes;

/* loop through the source doing vertical
 * slices starting at the left */
for (srcX = 0; srcX < srcWidth - 7; srcX += 8) {
/* bottom edge */
srcData = const1 + srcX / 8;
/* left edge */
dstData = dstAddr + srcX * dstRowBytes;
for (srcY = srcHeight - 1; srcY >= 0;
    srcY -= 8 ) {
rotateBlock(table, (unsigned char *)srcData,
srcRowBytes, (unsigned char *)dstData,
dstRowBytes);
srcData -= srcRowX8; /* next vert block */
++dstData; /* next horiz block */
}
}

/* handle the last partial row by calling
 * rotateLimitedBlock() */
if (remainder = (srcWidth % 8)) {
/* bottom edge */
srcData = const1 + srcX / 8;
/* left edge */
dstData = dstAddr + srcX * dstRowBytes;
for (srcY = srcHeight - 1; srcY >= 0;
    srcY -= 8 ) {
rotateLimitedBlock(table,
(unsigned char *)srcData, srcRowBytes,
(unsigned char *)dstData, dstRowBytes,
remainder);
srcData -= srcRowX8;
++dstData;
}
}
}

/*-----
Rotate an 8x8 block of pixels by treating the destination
block as 2 unsigned longs (64 bits!) and using a lookup
table to get a mask for the value of each row in the src
block. The table essentially stretches out the source byte
to 8 times its size and then ORs it into the destination
64bits. Since the bits are in the low bit of each byte,
```

iPHONE APP SHOWCASE

[TouchArcade Game of the Week: 'Vampire S...](#)

[SwitchArcade Round-Up: 'Dragon Quest Tre...](#)

['Total Football' is an Arcade-Style Socc...](#)

['JellyCar Worlds' is Out Now on Apple Ar...](#)

['Vampire Survivors' Mobile Review - Near...](#)

['Valiant Hearts: Coming Home' Revealed f...](#)

[Mini Metro's New Holiday Update Adds the...](#)

[Level Infinite announce December events...](#)

[Dead Cells: Return to Castlevania DLC Re...](#)

['Among Us' Hide and Seek Mode Update Com...](#)

[SEE ALL](#)

Price Scanner via MacPrices.net

[What's next for Apple? Mixed reality VR heads...](#)

[Mac Studio with M1 Max CPU back in stock at A...](#)

[Apple has 13" M2 MacBook Airs in stock with u...](#)

[Apple Pencil 2 on Christmas/Holiday sale for...](#)

[Midnight Apple Watch Series 8 \(45mm GPS\) on s...](#)

[The latest Holiday 14" Apple MacBook Pro sale...](#)

[Deal Alert! Amazon is offering \\$100 discounts...](#)

[Apple Magic Keyboard for 11" iPad Pro/10.9" i...](#)

[Lowest price of the season: 2022 Apple AirPods...](#)

[New! Details on Verizon's Christmas/Holiday p...](#)

[SEE ALL](#)

Jobs Board

[MacOS X / *Apple* Support Engineer - Royal...](#)

[Security Officer - *Apple* Store - NANA Reg...](#)

[*Apple* Electronic Repair Technician - PlanI...](#)

[Department Manager- *Apple* Store/Tech Shop...](#)

the destination has to be shifted one position to the left before each OR operation. The offset values are the difference to rowBytes. If we had a 64bit data type, this would be easier.

```
-----*/
void rotateBlock(unsigned long *table,
    unsigned char *src, long srcOffset,
    unsigned char *dst, long dstOffset)
{
    register unsigned long    resultLO, resultHi;
    short    x;

    resultLO = resultHi = 0;
    src += srcOffset * 7;
    /* compute the rotated result */
    for (x = 0; x < 8; ++x) {
        resultLO = resultLO << 1;
        resultHi = resultHi << 1;
        resultLO |= table[(src) >> 4];
        resultHi |= table[(src) & 0x0F];
        src -= srcOffset;
    }

    /* store the rotated result */
    *dst = (resultLO & 0xFF000000) >> 24;
    dst += dstOffset;
    *dst = (resultLO & 0x00FF0000) >> 16;
    dst += dstOffset;
    *dst = (resultLO & 0x0000FF00) >> 8;
    dst += dstOffset;
    *dst = resultLO & 0x000000FF;
    dst += dstOffset;
    *dst = (resultHi & 0xFF000000) >> 24;
    dst += dstOffset;
    *dst = (resultHi & 0x00FF0000) >> 16;
    dst += dstOffset;
    *dst = (resultHi & 0x0000FF00) >> 8;
    dst += dstOffset;
    *dst = resultHi & 0x000000FF;
}

/*-----
Same as other rotateBlock, except will do a
partial block. Useful to prevent the overwriting
of memory outside the bitmap!
-----*/
void rotateLimitedBlock(unsigned long *table,
    unsigned char *src, long srcOffset,
    unsigned char *dst, long dstOffset, short lines)
{
    register unsigned long    resultLO, resultHi;
    short    x;

    resultLO = resultHi = 0;
    src += srcOffset * 7;
    for (x = 0; x < 8; ++x) {
        resultLO = resultLO << 1;
```

```

    resultHi = resultHi << 1;
    resultLO |= table[(*src) >> 4];
    resultHi |= table[(*src) & 0x0F];
    src -= srcOffset;
}
*dst = (resultLO & 0xFF000000) >> 24;
dst += dstOffset; if (lines == 1) return;
*dst = (resultLO & 0x00FF0000) >> 16;
dst += dstOffset; if (lines == 2) return;
*dst = (resultLO & 0x0000FF00) >> 8;
dst += dstOffset; if (lines == 3) return;
*dst = resultLO & 0x000000FF;
dst += dstOffset; if (lines == 4) return;
*dst = (resultHi & 0xFF000000) >> 24;
dst += dstOffset; if (lines == 5) return;
*dst = (resultHi & 0x00FF0000) >> 16;
dst += dstOffset; if (lines == 6) return;
*dst = (resultHi & 0x0000FF00) >> 8;
dst += dstOffset; if (lines == 7) return;
*dst = resultHi & 0x000000FF;
}

//*****
// RotateBitMapClockwise
// By Jeff Mallett
//
// Rotates the significant bits of the srcBitMapPtr 90°
// clockwise and stores the result in dstBitMapPtr.
//*****

#define kHighShortBit0x8000
#define kBitsPerShort16

// Creates a short from bits in different rows of
// the source bitmap. Then copies this short into
// the destination bitmap.
#define COPY_TWO_BYTES(stopValue) \
    data = 0, bit = kHighShortBit; \
    do { \
        if (*srcPos & srcBit) data |= bit;\
        srcPos += srcRowShorts; \
    } while ( (bit >>= 1) != stopValue ); \
    *(dstPos++) = data

void RotateBitMapClockwise(Bitmap *srcBitMapPtr,
    Bitmap *dstBitMapPtr)
{
    register unsigned short data, *srcPos, bit;
    register int j;
    register unsigned short srcBit, *dstPos, stopBit;
    register int i;
    unsigned short *baseSrcPtr;
    // shorts per row of source
    const int srcRowShorts =
        srcBitMapPtr->rowBytes >> 1;
    const int numSrcRows =

```

```

srcBitMapPtr->bounds.bottom - srcBitMapPtr->bounds.top;

dstPos = (unsigned short *)dstBitMapPtr->baseAddr;
srcPos = baseSrcPtr =
    (unsigned short *)srcBitMapPtr->baseAddr;
srcBit = kHighShortBit;

for (i = srcBitMapPtr->bounds.right -
    srcBitMapPtr->bounds.left; i; --i) {
// Copy one column of source to a row of destination
for (j = numSrcRows / kBitsPerShort; j; --j) {
COPY_TWO_BYTES(0);
}
if (j = numSrcRows % kBitsPerShort) {
stopBit = kHighShortBit >> j;
COPY_TWO_BYTES(stopBit); // Final 2 bytes
}
// Prepare to copy next column of source
if ( !(srcBit >= 1) ) {
srcBit = kHighShortBit;
++baseSrcPtr;
}
srcPos = baseSrcPtr;
}
}
}

```

The Rules

Here's how it works: Each month there will be a different programming challenge presented here. First, you must write some code that solves the challenge. Second, you must optimize your code (a lot). Then, submit your solution to MacTech Magazine (formerly MacTutor). A winner will be chosen based on code correctness, speed, size and elegance (in that order of importance) as well as the postmark of the answer. In the event of multiple equally desirable solutions, one winner will be chosen at random (with honorable mention, but no prize, given to the runners up). The prize for the best solution each month is \$50 and a limited edition "The Winner! MacTech Magazine Programming Challenge" T-shirt (not to be found in stores).

In order to make fair comparisons between solutions, all solutions must be in ANSI compatible C (i.e., don't use Think's Object extensions). Only pure C code can be used. Any entries with any assembly in them will be disqualified. However, you may call any routine in the Macintosh toolbox you want (i.e., it doesn't matter if you use NewPtr instead of malloc). All entries will be tested with the FPU and 68020 flags turned off in THINK C. When timing routines, the latest version of THINK C will be used (with ANSI Settings plus "Honor 'register' first" and "Use Global Optimizer" turned on) so beware if you optimize for a different C compiler. All code should be limited to 60 characters wide. This will aid us in dealing with e-mail gateways and page layout.

The solution and winners for this month's Programmers' Challenge will be published in the issue two months later. All submissions must be received by the 10th day of the month printed on the front of this issue.

All solutions should be marked "Attn: Programmers' Challenge Solution" and sent to Xplain Corporation (the publishers of MacTech Magazine) via "snail mail" or preferably, e-mail - AppleLink: MT.PROGCHAL, Internet: progchallenge@xplain.com, and CompuServe: 71552,174. If you send via snail mail, please include a disk with the solution and all related files (including contact information). See page 2 for information on "How to Contact Xplain Corporation."